中国科学院大学 • 网络空间安全导论实验报告

箱子号 _____ 实验序号 ICS-exp01-01 实验专题 Linux 操作系统的使用与环境搭建

小组成员 （成员 1）陈曦

对应学号 2025K8009970002

最后修改日期 2026 年 3 月 12 日

1 摘要

本实验采用 WSL2+Ubuntu 搭建环境，旨在掌握 Linux 基础操作、探究核心机制并实现综合应用。完成 10 项基础命令操作，通过 man 手册、跨语言验证及系统测试深入研究了信号机制与命令行参数限制。综合应用 Docker 部署与 Wine 运行，解决了网络超时、依赖缺失等实际问题。借助 AI 工具辅助分析，通过实操验证了 Linux 核心特性，理解了相关工具的实现逻辑，为后续安全实验奠定基础。

2 实验环境

1. 硬件环境

项目名称	具体配置
设备型号	Lenovo Xiaoxin air15 2020
处理器	AMD Ryzen 7 4800U
内存	16GB（WSL2 自动管理内存占用）
存储	512GB NVMe 固态硬盘

2. 软件环境

项目名称	具体配置
主操作系统	Windows 11 家庭版（25H2 内核版本号 26200.7922）
Linux 虚拟化环境	WSL2（Windows Subsystem for Linux 2，内核版本 5.15.90.1）
Linux 发行版	Ubuntu 24.04 LTS（64 位）
核心工具	GCC 13.3.0、Python 3.12.3、Vim 9.1、Docker Desktop 4.63.0 (220185) Wine 11.0 Stable、Winetricks 0.0.1-20240105
辅助工具	Deepseek R1、浏览器（Microsoft Edge）

3 Linux 基本操作实验过程操作步骤

1. 构建一个名为 `lab1` 的文件夹

```
mkdir lab1
```

2. 进入 `lab1` 目录

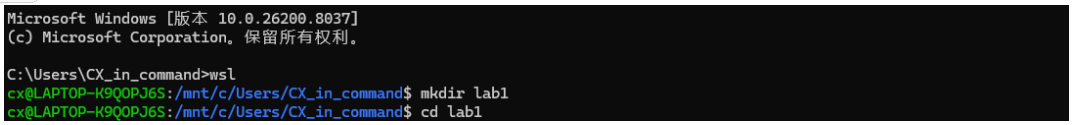


图 1: 构建并进入文件夹

其中 `cd [ DIRECTORY ]` 是常用的命令，还有：

```
cd .   //这里的 .  指代的是当前文件夹
cd ..  //这里的 .. 指代的是父文件夹
cd -   //这里的 -  指代的是上次所访问的目录
cd ~   //这里的 ~  指代的是根目录
```

3. 创建一个 C 源码，编译并运行一个 hello,world：

```c
// Hello world 的 C 语言实现:
#include<stdio.h>
int main(){
    printf("Hello,world");
    return 0;
}
```



图 2: 创建 Hello，world 的 C 源码，编译运行

4. 用 Python 算第 100 个斐波那契数并存在文件中：

```python
# 计算斐波那契数列的 Python 程序实现
def fibonacci(n):
    if n <= 0:
        print(" 数列 index 非法")
        return None
```

```python
    dp = [0] * (n+1)
    dp[0] = 0
    if n >= 1:
        dp[1] = 1
    for i in range(2, n + 1):
        dp[i] = dp[i - 1] + dp[i - 2]
    return dp[n]

if __name__ == "__main__":
    result = fibonacci(100)
    print(result)
```

```
cx@LAPTOP-K9QOPJ6S:/mnt/c/Users/CX_in_command/lab1$ vim solve.py
cx@LAPTOP-K9QOPJ6S:/mnt/c/Users/CX_in_command/lab1$ python3 solve.py > answer.txt
```

图 3: 用 Python 算第 100 个斐波那契数并存在文件中

这里解释一下其中的 `>` 操作符以及一些其他的操作符

操作符	含义
`>`	写入并覆盖文件
`»`	追加写入
`;`	一行多个的分割符

5. 拷贝/etc/os-release 文件到当前目录：

```
cx@LAPTOP-K9QOPJ6S:/mnt/c/Users/CX_in_command/lab1$ cp /etc/os-release .
```

图 4: 拷贝/etc/os-release 文件到当前目录

6. 将/etc/fstab 的 base64 编码写入文件：

```
cx@LAPTOP-K9QOPJ6S:/mnt/c/Users/CX_in_command/lab1$ base64 /etc/fstab > fstab_base64.txt
```

图 5: 将/etc/fstab 的 base64 编码写入文件 `fstab_base64.txt`

7. 查看 /proc/self/maps 并将结果保存到文件中：

```
cx@LAPTOP-K9QOPJ6S:/mnt/c/Users/CX_in_command/lab1$ cat /proc/self/maps | tee maps.txt
5715c6b93000-5715c6b95000 r--p 00000000 08:30 11592                      /usr/bin/cat
5715c6b95000-5715c6b9a000 r-xp 00002000 08:30 11592                      /usr/bin/cat
5715c6b9a000-5715c6b9c000 r--p 00007000 08:30 11592                      /usr/bin/cat
5715c6b9c000-5715c6b9d000 r--p 00008000 08:30 11592                      /usr/bin/cat
5715c6b9d000-5715c6b9e000 rw-p 00009000 08:30 11592                      /usr/bin/cat
5715ee8b1000-5715ee8d2000 rw-p 00000000 00:00 0                          [heap]
7b4411fa7000-7b4412000000 r--p 00000000 08:30 17648                      /usr/lib/locale/C.utf8/LC_CTYPE
7b4412000000-7b4412028000 r--p 00000000 08:30 188091                     /usr/lib/x86_64-linux-gnu/libc.so.6
7b4412028000-7b44121b0000 r-xp 00028000 08:30 188091                     /usr/lib/x86_64-linux-gnu/libc.so.6
7b44121b0000-7b44121ff000 r--p 001b0000 08:30 188091                     /usr/lib/x86_64-linux-gnu/libc.so.6
7b44121ff000-7b4412203000 r--p 001fe000 08:30 188091                     /usr/lib/x86_64-linux-gnu/libc.so.6
7b4412203000-7b4412205000 rw-p 00202000 08:30 188091                     /usr/lib/x86_64-linux-gnu/libc.so.6
7b4412205000-7b4412212000 rw-p 00000000 00:00 0
7b441223b000-7b4412260000 rw-p 00000000 00:00 0
7b4412265000-7b4412266000 r--p 00000000 08:30 17654                      /usr/lib/locale/C.utf8/LC_NUMERIC
7b4412266000-7b4412267000 r--p 00000000 08:30 20675                      /usr/lib/locale/C.utf8/LC_TIME
7b4412267000-7b4412268000 r--p 00000000 08:30 16413                      /usr/lib/locale/C.utf8/LC_COLLATE
7b4412268000-7b4412269000 r--p 00000000 08:30 17652                      /usr/lib/locale/C.utf8/LC_MONETARY
7b4412269000-7b441226a000 r--p 00000000 08:30 17651                      /usr/lib/locale/C.utf8/LC_MESSAGES/SYS_LC_MESSAGES
7b441226a000-7b441226b000 r--p 00000000 08:30 20673                      /usr/lib/locale/C.utf8/LC_PAPER
7b441226b000-7b441226c000 r--p 00000000 08:30 17653                      /usr/lib/locale/C.utf8/LC_NAME
7b441226c000-7b441226d000 r--p 00000000 08:30 14750                      /usr/lib/locale/C.utf8/LC_ADDRESS
7b441226d000-7b441226e000 r--p 00000000 08:30 20674                      /usr/lib/locale/C.utf8/LC_TELEPHONE
7b441226e000-7b441226f000 r--p 00000000 08:30 17650                      /usr/lib/locale/C.utf8/LC_MEASUREMENT
7b441226f000-7b4412276000 r--s 00000000 08:30 188079                     /usr/lib/x86_64-linux-gnu/gconv/gconv-modules.cache
7b4412276000-7b4412277000 r--p 00000000 08:30 17649                      /usr/lib/locale/C.utf8/LC_IDENTIFICATION
7b4412277000-7b4412279000 rw-p 00000000 00:00 0
7b4412279000-7b441227a000 r--p 00000000 08:30 188088                     /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7b441227a000-7b44122a5000 r-xp 00001000 08:30 188088                     /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7b44122a5000-7b44122af000 r--p 0002c000 08:30 188088                     /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7b44122af000-7b44122b1000 r--p 00036000 08:30 188088                     /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7b44122b1000-7b44122b3000 rw-p 00038000 08:30 188088                     /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7ffe3073a000-7ffe3075b000 rw-p 00000000 00:00 0                          [stack]
7ffe307b2000-7ffe307b6000 r--p 00000000 00:00 0                          [vvar]
7ffe307b6000-7ffe307b8000 r-xp 00000000 00:00 0                          [vdso]
```

图 6: 查看 /proc/self/maps 并将结果保存到文件 `maps.txt`

这里解释一下 Linux 操作系统中比较特有的管道操作符 `|`

他允许将前一个命令的输出作为后一个命令的输入

`tee` 命令的使用 `tee [OPTION] [I/O]`

8. 将/bin/sh 的十六进制数据存储到文件中：

```
cx@LAPTOP-K9QOPJ6S:/mnt/c/Users/CX_in_command/lab1$ hexdump -C /bin/sh > sh_hexdump.txt
```

图 7: 将/bin/sh 的十六进制数据存储到文件 `sh_hexdump.txt`

9. 返回上一级目录并打包 lab1 目录：
这里解释一下 tar 工具的使用，tar 工具 (同时配合 gzip 工具使用) 类似于 Windows 下的 **7-Zip** 或者 **WinRAR**，不过相比他们不同的是 tar 是一个**命令行应用 (CommandLine Application)**。tar 工具的使用参数见下表

参数	解释
-c	创建一个归档文件.tar 同时配合-f 参数使用
-v	显示创建的详细过程
-z	告诉 tar 创建归档文件的时候要同时压缩
-r	将一个文件添加进这个归档文件.tar 中
-x	将一个.tar 文件解包成单独的文件
-C	指定解压目录 (需要提前创建目录)
-f	指定输出的文件名
-t	列出一个归档包或压缩包的内容

注：tar 需要解压的话可以自动识别不需要指定参数

```
cx@LAPTOP-K9QOPJ6S:/mnt/c/Users/CX_in_command/lab1$ cd ..
cx@LAPTOP-K9QOPJ6S:/mnt/c/Users/CX_in_command$ tar -czvf lab1-chenxi.tar.gz lab1/
lab1/
lab1/answer.txt
lab1/fstab_base64.txt
lab1/hello
lab1/hello.c
lab1/maps.txt
lab1/os-release
lab1/sh_hexdump.txt
lab1/solve.py
cx@LAPTOP-K9QOPJ6S:/mnt/c/Users/CX_in_command$
```

图 8: 返回上一级目录并打包 lab1 目录

4 Linux 问题探究

问题 1 *Linux* 有哪些信号？你可以使用哪些方法来让一个程序（*C* 程序、*Python* 程序、*shell* 程序均可）触发不同的信号？进程收到哪些信号退出时，会在 *shell* 中显示进程收到信号的信息？

分析 1 研究过程：通过查询网络资料 *[1]* 与 *Linux manual(手册)[2]*，得知 *Linux* 信号（*Signal*）是一种进程间通信（*IPC, Inter-Process Communication*）机制，用于通知进程某个事件发生。最典型的就是经常在 *VSCode* 的控制台中按 *Ctrl + C* 结束刚编译好的程序

联网搜索得知：通过查阅 *man 7 signal* 文档可以了解到 *Linux* 系统中信号（*Signal*）的类型、默认行为以及信号处理机制。需要注意的是，*Linux* 的 *man* 手册按章节分类，同一名称可能在不同章节中对应不同内容。例如 *man signal* 通常会打开 *signal* 系统调用或库函数的说明，而 *man 7 signal* 则提供了 *Linux* 信号机制的总体介绍。

```
Standard signals
       Linux supports the standard signals listed below.  The second column of the table indicates which standard (if
       any)  specified  the signal: "P1990" indicates that the signal is described in the original POSIX.1-1990 stan-
       dard; "P2001" indicates that the signal was added in SUSv2 and POSIX.1-2001.

       Signal       Standard   Action   Comment
       ──────────────────────────────────────────────────────────────────────
       SIGABRT      P1990      Core     Abort signal from abort(3)
       SIGALRM      P1990      Term     Timer signal from alarm(2)
       SIGBUS       P2001      Core     Bus error (bad memory access)
       SIGCHLD      P1990      Ign      Child stopped or terminated
       SIGCLD         -        Ign      A synonym for SIGCHLD
       SIGCONT      P1990      Cont     Continue if stopped
       SIGEMT         -        Term     Emulator trap
       SIGFPE       P1990      Core     Floating-point exception
       SIGHUP       P1990      Term     Hangup detected on controlling terminal
                                        or death of controlling process
       SIGILL       P1990      Core     Illegal Instruction
       SIGINFO        -                 A synonym for SIGPWR
       SIGINT       P1990      Term     Interrupt from keyboard
       SIGIO          -        Term     I/O now possible (4.2BSD)
       SIGIOT         -        Core     IOT trap. A synonym for SIGABRT
       SIGKILL      P1990      Term     Kill signal
       SIGLOST        -        Term     File lock lost (unused)
       SIGPIPE      P1990      Term     Broken pipe: write to pipe with no
                                        readers; see pipe(7)
       SIGPOLL      P2001      Term     Pollable event (Sys V);
                                        synonym for SIGIO
       SIGPROF      P2001      Term     Profiling timer expired
       SIGPWR         -        Term     Power failure (System V)
       SIGQUIT      P1990      Core     Quit from keyboard
       SIGSEGV      P1990      Core     Invalid memory reference
       SIGSTKFLT      -        Term     Stack fault on coprocessor (unused)
       SIGSTOP      P1990      Stop     Stop process
       SIGTSTP      P1990      Stop     Stop typed at terminal
       SIGSYS       P2001      Core     Bad system call (SVr4);
                                        see also seccomp(2)
       SIGTERM      P1990      Term     Termination signal
       SIGTRAP      P2001      Core     Trace/breakpoint trap
       SIGTTIN      P1990      Stop     Terminal input for background process
       SIGTTOU      P1990      Stop     Terminal output for background process
       SIGUNUSED      -        Core     Synonymous with SIGSYS
       SIGURG       P2001      Ign      Urgent condition on socket (4.2BSD)
       SIGUSR1      P1990      Term     User-defined signal 1
       SIGUSR2      P1990      Term     User-defined signal 2
       SIGVTALRM    P2001      Term     Virtual alarm clock (4.2BSD)
       SIGXCPU      P2001      Core     CPU time limit exceeded (4.2BSD);
                                        see setrlimit(2)
       SIGXFSZ      P2001      Core     File size limit exceeded (4.2BSD);
                                        see setrlimit(2)
       SIGWINCH       -        Ign      Window resize signal (4.3BSD, Sun)
Manual page signal(7) line 186 (press h for help or q to quit)
```

图 9: 列出 Linux 中所有信号

同时使用 `man signal` 我们可以看到 *Linux* 系统下程序如何使用 *signal* 相关函数处理信号

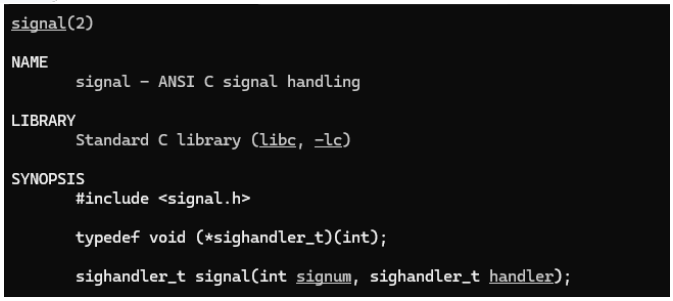


图 10: Linux 系统中对 Signal 的 C 标准库的说明

很自然的，我们会想到 *C* 程序的信号触发方式，

```c
// 5 秒后程序给自己发送 SIGINT
#include <signal.h>
#include <stdio.h>
#include <unistd.h>
int main(){
    printf("PID = %d \n", getpid());
    sleep(5);
    raise(SIGINT);
    printf("If this program were still running, it would print this
    ↪  message.\n");
    return 0;
}
```

运行如下

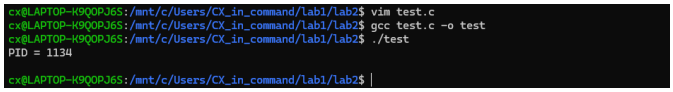


图 11: C 程序触发信号 SIGINT(类似于 `Ctrl + C`)

除了程序触发外，还有手动输入的方式如:

- *Ctrl + C* 触发 *SIGINT*
- *Ctrl + * 触发 *SIGQUIT*
- *Ctrl + Z* 触发 *SIGTSTP*

同时还有一种内核在检测到异常或错误时抛出 *Signal*(比如检测到除以零，或者是某些 *I/O* 异常)

另外，当程序因为信号终止，*shell* 有时会显示提示。例如:

```
Segmentation fault (core dumped)
```

常见情况:

信号	shell 显示
SIGSEGV	Segmentation fault
SIGABRT	Aborted
SIGFPE	Floating point exception
SIGILL	Illegal instruction
SIGQUIT	Quit
SIGPIPE	Broken pipe

例子：

```c
// C 程序触发信号 SIGSEGV，shell 显示提示
#include <stdio.h>
int main(){
    int *p = NULL;
    *p = 5;
}
```

运行如下：

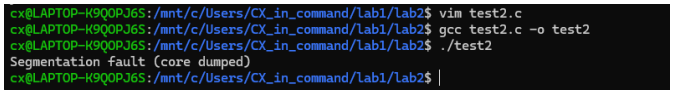


图 12: C 程序触发信号 SIGSEGV，shell 显示提示

实验结论

- Linux 信号是进程间通信机制之一。
- 可以通过 kill、键盘、系统调用触发信号。
- 不同信号默认行为不同。
- 当程序因异常信号退出时，shell 会输出错误信息。

问题 2 一个程序的命令行参数是什么？有什么限制？它在一个程序（C 程序、Python 程序、shell 程序均可）中是如何使用的？

分析 2 这一点可以从我们日常使用的命令行工具找到答案。例如我们常使用的

```
ls -l /home
```

其中的 `-l /home` 部分称作命令行参数，相当于将程序的 Input 前置。对于 Linux 系统原生支持的 C 语言而言，命令行参数在于 `main()` 函数的可选参数中

```c
int main(int argc, char *argv[])
// 在许多古早的代码如 ls 工具中也写成
int main(int argc, char **argv)
```

参数	含义
argc	参数数量
argv	参数字符串数组

示例程序：

```c
#include<stdio.h>
int main(int argc, char *argv[]){
    printf("argc = %d\n", argc);
    for(int i = 0; i < argc; i++){
        printf("argv[%d] = %s\n", i, argv[i]);
    }
    return 0;
}
```

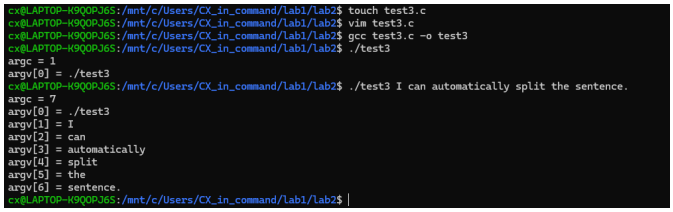


图 13: 获取命令行参数

其中 *argv[0]* 通常表示程序自身的名称。剩下的从 *1 -> argc-1* 都是 *OS* 主动将参数以空格拆分得到的多个字符串

而对于其他语言：

在 *Python* 中可以通过 *sys.argv* 获取命令行参数。

示例代码：

```python
import sys

if __name__ == "__main__":
    print(len(sys.argv))
    for arg in sys.argv:
        print(arg)
```

运行示例：

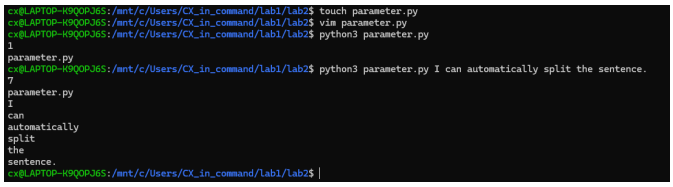


图 14: Python 程序获取命令行参数

在 *Shell* 脚本中，命令行参数可以通过 `$1`、`$2` 等变量访问，`$#` 表示参数数量。

示例脚本：

```
#!/bin/bash
echo "first argument: $1"
echo "second argument: $2"
echo "number of arguments: $#"
```

运行示例：

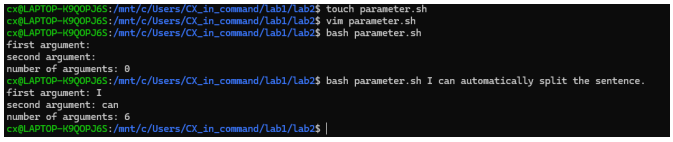


图 15: Shell 脚本获取命令行参数

而这项功能的最大作用就是开发所谓的命令行式应用（*Command-Line Application*）。这种应用没有 *GUI(图形界面)*，资源占用少，效率高；通过解析不同组合的参数实现特定的功能。或者某种意义上也叫做脚本

而当然，对于这种 *C* 风味的字符串数组，一般都有一个固定的上限长度。该限制由内核参数 `ARG_MAX` 决定。可以通过以下命令查看该限制：

```
getconf ARG_MAX
```

在实验环境中得到的结果为：

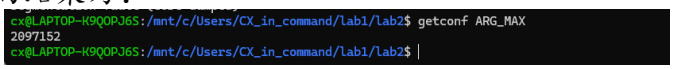


图 16: 实验室得到的 ARG_MAX 参数

同时我们使用 *shell* 命令构造超长参数，此时内核拒绝创建新进程，内核抛出错误：

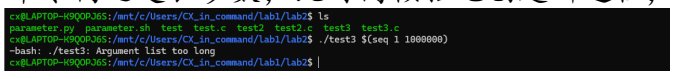


图 17: 命令行参数超过 ARG_MAX 时

实验总结

命令行参数是程序启动时从命令行传入的参数，用于控制程序的运行行为。

在不同语言中访问命令行参数的方法不同：

- *C* 语言：通过 `argc` 和 `argv`
- *Python*：通过 `sys.argv`
- *Shell*：通过 `$1`、`$2` 等变量访问参数值，`$#` 访问参数数量

同时，*Linux* 系统对命令行参数的总长度存在限制，该限制由系统参数 `ARG_MAX` 决定。

5 Linux 综合应用

应用 1 *安装 Docker 并运行 Hello World*

实验 1 首先，由于我使用的是 *WSL2* 的环境，考虑到希望同时在主机 *Windows* 系统及多个 *WSL* 的容器中方便地使用 *Docker*，因此选择在主机 *Windows* 系统上安装 *Docker Desktop* 应用，通过挂载的方式分配给 *Linux* 虚拟机使用。首先安装 *Docker Desktop*. 之后全程采用 *GUI* 操作，十分方便 *(PS: ~~我这是不是算走捷径~~)*

然后启动应用，来到 *Setting* → *Resources* → *WSL Integration* 此处，勾选启用 *Docker* 与 *WSL* 的集成，并选择需要启用 *Docker* 的特定 *Linux* 发行版。

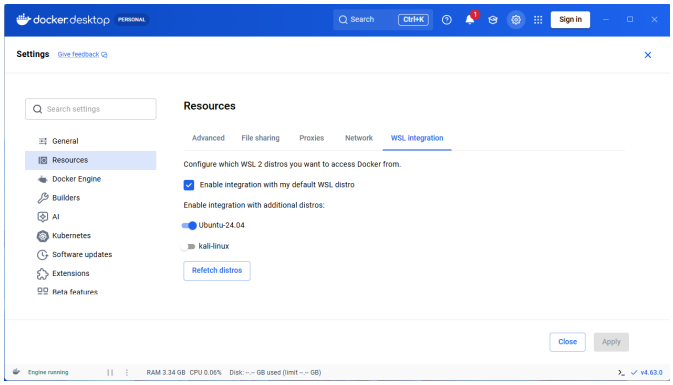


图 18: 为 WSL 启用 Docker

此时，*Docker* 自动配置完成，在 *WSL* 的 *Ubuntu* 系统的 *Bash* 中输入

```
1 docker version
```

Shell 输出提示：

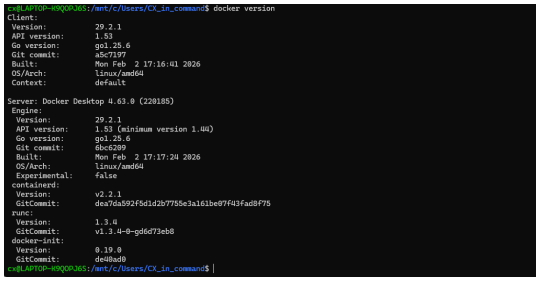


图 19: 在 Shell 中输入 docker version

然后开始访问 *Docker Hub* 下载官方的 *hello-world* 容器运行。但是第一次运行时发现，*Shell* 界面长时间未响应，因此选择 *Ctrl + C* 退出。



图 20: 下载 hello-world 容器但长时间未响应

参考 *AI* 辅助分析，尝试运行

```
systemctl status docker
```

检查 *docker* 守护进程运行状态,是不是 *Docker* 进程被 *kill* 掉了,但是提示 *docker.service Not Found*

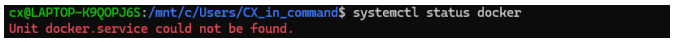


图 21: 查询 Docker 守护进程状态

分析本机安装的特殊性发现：本机 *Docker* 的主进程事实上是在 *Windows* 上的，*Linux* 上的只是一个挂载的 *Docker* 套皮。所以 *systemd* 事实上不会管理到本机的 *docker* 程序。也就是说问题的根源还是在网络上。受到 *GFW(Great Firewall)* 影响，从国内访问国外的 *Docker Hub* 网站下载容器几乎是被阻塞的，所以需要一些"魔法科技"(" 梯子")。挂上" 梯子" 之后，就成功从 *Docker Hub* 上下载并成功运行了官方的 *hello-world* 容器，同时产生了合适的输出了

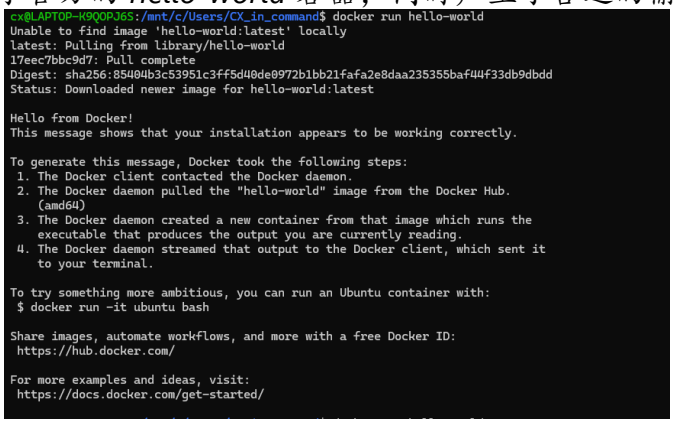


图 22: 运行 hello-world 容器

对 *Docker* 的理解：不只是虚拟化

说实话，在动手安装之前，我以为 *Docker* 就和平时用的虚拟机差不多：装一个系统环境，在里面跑应用。但 *Docker* 事实上比我想象的要“轻”得多——与其说是虚拟机，*Docker* 更像是对 *Linux* 操作系统内核功能的二次封装。

具体来说，*Linux* 内核早就支持把进程隔离开，还能限制每个进程对 *CPU*、内存的占用。这些功能以前是给系统管理员做高级调优用的，但 *Docker* 把它们用起来了：每一个容器其实就是一个被“隔离”起来的普通进程，它有自己的文件系统、网络、进程编号，但用的还是同一个 *Linux* 内核。所以容器启动特别快，占用的资源也少，不像虚拟机还要模拟一整台电脑。

Docker 通过对运行环境的打包，实现了‘一次构建，随处运行’。从某种视角看，它和 *JVM* 都提供了**环境抽象**：*JVM* 抽象了操作系统和硬件，让 *Java* 程序跨平台运行；而 *Docker* 抽象了操作系统发行版和依赖，让应用容器能在不同 *Linux* 环境下一致运行。虽然两者的实现原理完全不

同：*JVM* 是语言虚拟机，*Docker* 是操作系统级虚拟化，直接使用宿主机内核。

应用 2 使用 *Wine* 运行 *Windows* 程序

实验 2 参考知乎 *Linux* 中国的解释文档：*[3]如何在 Ubuntu 中安装 Wine | Linux 中国*
同时参考另一个 *CSDN Blog* 文档：*[4]Ubuntu24 使用 Wine 运行 Windows 程序安装微信*

- 选择从 *wine* 的存储库安装最新的版本

  1. 首先输入下面的代码使得在我的发行版上添加 *32* 位架构的支持

     ```
     sudo dpkg --add-architecture i386
     ```

  2. 下载和添加存储库密钥:

     ```
     sudo mkdir -pm755 /etc/apt/keyrings
     sudo wget -O /etc/apt/keyrings/winehq-archive.key
     ↪ https://dl.winehq.org/wine-builds/winehq.key
     ```

  3. 现在，下载 *WineHQ* 源文件文件。由于我使用的 *Linux* 发行版是 *Ubuntu-24.04*，此时参考 *CSDN Blog* 中的添加方式：*(*注意不同的 *Ubuntu* 版本对应的 *winehq-*代号不同，不可以随意使用*)*

     ```
     sudo wget -NP /etc/apt/sources.list.d/
     ↪ https://dl.winehq.org/wine-builds/ubuntu/dists/noble/winehq-noble.sources
     ```

     在完成后，更新软件包信息 *(*必须要干*)* 和安装 *Wine* 的稳定版本软件包。

     ```
     sudo apt update
     sudo apt install --install-recommends winehq-stable
     ```

     安装 *Wine* 的辅助工具 *Winetricks*:

     ```
     udo apt install winetricks
     ```

- 初始化 *Wine* 配置

  初始化 *wine* 环境，使用以下命令安装 *wine-mono* 组件来支持.*NET* 应用：

  ```
  winecfg
  ```

使用 *Winetricks* 检查 *wine-mono* 和 *wine-gecko* 是否安装成功，执行如下命令，然后依次选择 `选择默认的 wine 容器 → 运行卸载程序`，在弹出的窗口中如果发现了 *wine-mono* 和 *wine-gecko*，则说明 *wine-mono* 和 *wine-gecko* 安装成功。:

```
winetricks
```

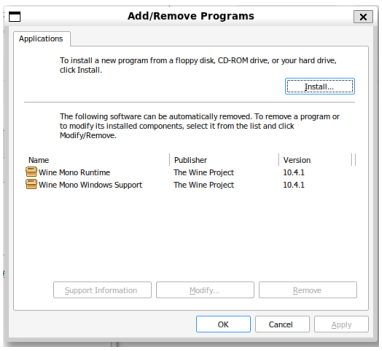


图 23: 使用 winetricks 检查自动安装程序有无安装上面两个程序

发现仍然缺少 *wine-gecko*，下面进入清华开源下载站下载。打开清华大学开源软件镜像站 *wine-gecko* 目录：清华下载站下载，找到最新版 *wine-gecko*（非 *rc* 版），下载其 *msi* 格式的安装包（需要下两个，*32* 位 *x86* 的和 *64* 位 *x86_64* 的）：下载好后打开 *winetricks*，然后选择上图中的 *Install* 选项选中对应的.*msi* 程序安装，等待一段时间后安装完成

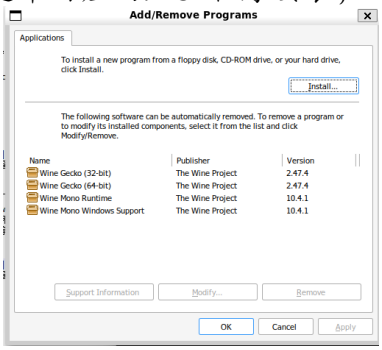


图 24: 使用 winetricks 检查 wine-gecko 是否安装完整

- **安装常用配置**

  安装常用字体，输入下列命令：

```
winetricks allfonts corefonts cjkfonts
```

  安装 *DirectX* 提升兼容性和应用性能

```
winetricks d3dx9 d3dx10
```

  安装 *VC++2022* 运行库

```
1  winetricks vcrun2022
```

- 下面以 *7-Zip* 为例子来展示 *wine* 的使用：

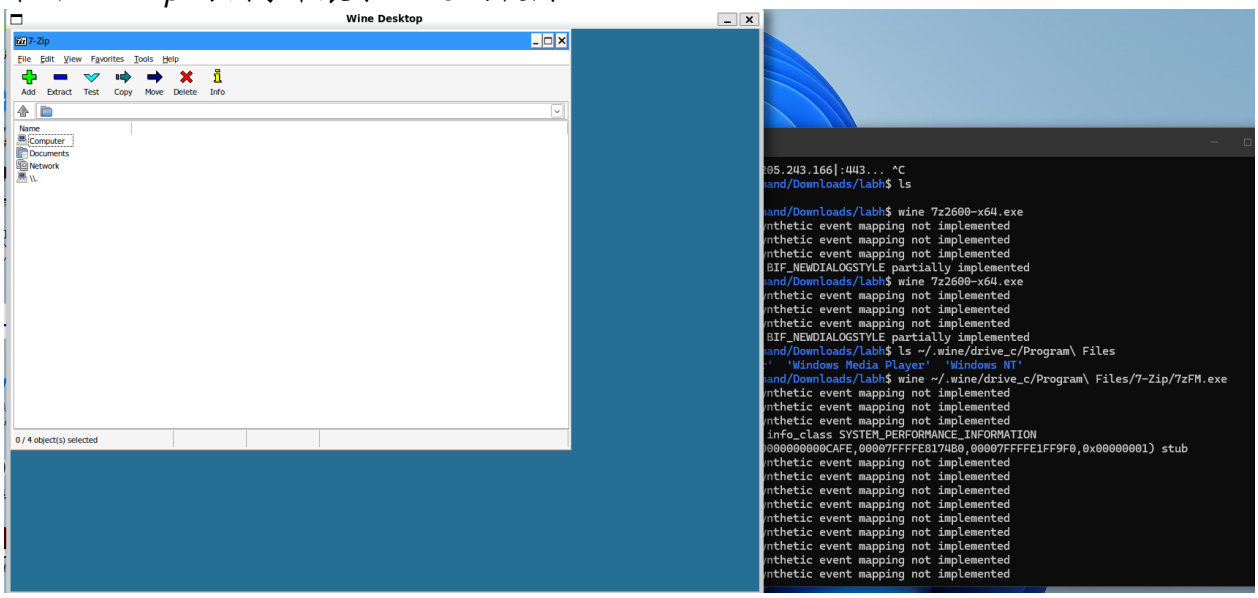


图 25: 使用 wine 启动 7-Zip

实验总结

通过本实验，我在 *Linux* 环境中成功安装并配置了 *Wine*，并使用 *Wine* 成功运行了 *Windows* 程序。实验过程中，我了解了 *Wine* 的工作原理:

> *Note*
>
> 即通过实现 *Windows API* 接口，使 *Windows* 应用程序能够在 *Linux* 系统上运行，而无需安装完整的 *Windows* 操作系统。

相比虚拟机方案，*Wine* 具有资源消耗低、运行效率高等优势，但其兼容性仍然有限，一些复杂的 *Windows* 程序可能无法正常运行。总体而言，*Wine* 是一种轻量级且高效的跨平台解决方案，在 *Linux* 环境中运行 *Windows* 应用具有重要意义。

6 参考文献与附录

参考文献

[1] Linux–信号（万字详解！超完整！）[N]https://blog.csdn.net/2301_76618602/article/details/139576092

[2] Linux man-pages project. (2023). *signal(7) —Linux manual page*.

[3] 如何在 Ubuntu 中安装 Wine | Linux 中国 [N] https://zhuanlan.zhihu.com/p/645456382

[4] Ubuntu24 使用 Wine 运行 Windows 程序安装微信 [N]
https://blog.csdn.net/XiaoYuHaoAiMin/article/details/141901660

AI 辅助呈现

运行 docker run hello-world 时长时间卡在“Unable to find image 'hello-world:latest' locally”，然后按了 Ctrl+C 终止，分析一下原因

已思考（用时 15 秒）

你在运行 `docker run hello-world` 时卡在“Unable to find image 'hello-world:latest' locally”，然后按了 Ctrl+C 终止。这通常是因为 Docker 尝试从 Docker Hub 拉取镜像时网络不通或超时。下面提供几步排查和解决方法：

---

1. 检查 Docker 守护进程是否运行

在终端执行：

bash 复制 下载

```
sudo systemctl status docker
```

- 如果看到 `active (running)`，说明守护进程正常。
- 如果未运行，启动它：

bash 复制 下载

```
sudo systemctl start docker
```

- 如果提示 `systemctl` 命令不存在（在 WSL 中未使用 systemd），可以尝试：

给 DeepSeek 发送消息

深度思考 智能搜索

内容由 AI 生成，请仔细甄别